

AI Neta — Production-First Redesign Plan

Status: Greenfield implementation in progress — re-audited against `AGENTS.md`

Date: 10 August 2026

Scope: Greenfield redesign based on the product requirements, low-literacy research, mass-scale architecture research, the multi-agent research note, and the repository's production-first engineering rules.

1. Executive decision

AI Neta should be rebuilt as a production-first civic accountability platform with:

1. One consistent citizen-facing AI Neta voice.
2. Multiple narrow handlers behind that voice, each with a small tool set and explicit safety boundary.
3. A deterministic complaint state machine for evidence, verification, routing, SLA tracking, escalation, and closure.
4. Durable workflows and append-only events as the source of truth for follow-through.
5. A native, offline-tolerant citizen experience alongside a lightweight web and channel-based experience.
6. An India-resident deployment reference architecture with replaceable provider adapters.
7. The simplest production-capable implementation path: a Python/FastAPI application with Agno for bounded AI orchestration, rather than a second AI runtime only to support a different framework language.

The key design principle is:

The model may understand language and phrase a response. It must not decide whether evidence is valid, whether a complaint should escalate, whether a department has replied, or whether a case is resolved.

The system should begin as a modular platform with a small number of deployable units, not as dozens of independently operated microservices. Event contracts, workflow boundaries, and provider interfaces will be designed from day one so individual components can scale out without a rewrite. "Mass scale" is a capacity and reliability target; it is not a reason to introduce every distributed-systems component before the first vertical slice proves the need.

2. Source material and non-negotiable requirements

This plan treats the following as the authoritative product and architecture inputs:

- [AI Neta PRD](#)
- [Low-literacy and first-time-user research](#)
- [Mass-scale technology research](#)
- [Multi-agent architecture research](#)

The redesign must preserve these product truths:

- AI Neta is a citizen representative, not a generic chatbot, activism tool, or political product.
- Complaint intake is Pan-India from day one.
- Full outbound automation is restricted to verified active execution zones, beginning with Madhya Pradesh.
- Complaints without verified contacts are accepted and held in **Mapping in progress** state.
- Silence and delay are first-class evidence.

- Escalation is rule-based and follows L1 → L2 → L3 → L4.
- A department reply does not reset an SLA unless the underlying issue is actually fixed.
- Closure requires evidence and citizen confirmation.
- Public information is neutral, aggregated where necessary, consent-aware, and exportable.
- Every core citizen interaction must be completable by a person who cannot read, using voice and tap-based interactions.

Decision baseline for this redesign

The following choices are now fixed for the current planning and implementation baseline. They remove ambiguity from the build without pretending that the external production approvals have already happened:

1. **Feature-phone location:** do not infer precise GPS from SMS/IVR. A feature-phone or assisted intake is accepted through a callback/operator flow that obtains a locality and later confirms the location before any zone-specific outbound action. It remains a lower-confidence or `mapping_in_progress` case until that confirmation exists.
2. **Identity verification:** DigiLocker is the primary citizen-verification path. The implementation uses the official Requester/OAuth/consent boundary, stores a verification reference and minimum approved claims rather than raw documents by default, and remains sandboxed until partner onboarding, transport approval, and retention/security review are complete. See the [DigiLocker Requester integration guidance](#) and [API Setu resource center](#).
3. **Public disclosure:** the default is private case handling with aggregated transparency. Case-level publication requires a separate, explicit, versioned citizen consent and policy-approved redaction; verification never implies permission to publish a citizen's identity, evidence, or exact location.
4. **Operations data:** live hierarchy, contacts, calendars, and SLA data are intentionally skipped for now. Synthetic MP fixtures remain available for local and controlled testing, and no blind outbound dispatch is enabled.
5. **Telecom specifics:** provider selection and telecom integration are intentionally skipped for now. Channel interfaces and deterministic fakes remain in place so later activation does not change domain behavior.
6. **Budget:** use a small managed-service launch profile with strict monthly spend ceilings, provider quotas, token/media limits, alerts, and scale-up gates based on measured usage. Do not commit to EKS, Kafka, dedicated GPUs, multi-region infrastructure, or large always-on model capacity before the usage and cost evidence justifies it.
7. **Duplicate and collective issues:** satisfy the PRD requirement with a non-destructive, deterministic cluster-candidate boundary. Individual complaints remain independently owned, auditable, routable, and closable; the system must not silently merge, suppress, or transfer a citizen's case. Supporter counts and public cluster views are policy-controlled projections, not model decisions. Exact spatial, category, time-window, abuse, and public disclosure thresholds remain versioned configuration until product/privacy review approves them.

The remaining external approvals are activation gates, not reasons to block local implementation: DigiLocker Requester onboarding and legal review, approved identity claims/retention, closure/disclosure policy sign-off, live routing authority data, and the finance-owned monthly ceiling.

2.1 Re-audit decisions from `AGENTS.md`

The repository instructions make low integration friction an explicit architecture requirement. Applying that rule changes several previous recommendations:

- **AI orchestration:** choose Agno because it is Python-native, fits the FastAPI application boundary, and provides agents, workflows, structured Pydantic output, sessions, guardrails, and tracing without adding a TypeScript AI service. See the [Agno agent overview](#), [workflow overview](#), and [structured output documentation](#).
- **Application runtime:** retain FastAPI as the primary API/application runtime and reorganize it into presentation, application, domain, infrastructure, and operations boundaries. A NestJS rewrite would add migration and runtime cost without a product requirement that demands it.

- **Durable business workflow:** use Temporal for multi-hour/day complaint SLAs, retries, signals, and escalation timers. Agno workflows handle bounded conversational orchestration; they do not become the complaint system of record.
- **Event transport:** create versioned events and a transactional outbox first. Add a Kafka-compatible broker when measured throughput, replay, or consumer isolation requires it. The outbox/event interface keeps that scale path open without making Kafka a local or first-slice dependency.
- **AI authority:** the PRD's "AI Decision Core" is interpreted as AI-generated signals and recommendations. Final approval, priority thresholds, routing, escalation, public disclosure, and closure remain deterministic policy decisions.
- **Deferred external inputs:** real operations contacts and telecom provider details are intentionally out of the current build scope. The budget is limited, so the baseline favors managed services and bounded provider calls, with quotas and alarms required before paid production traffic. These inputs remain activation gates only where the live system would otherwise send messages or incur uncontrolled spend.

This is a correction to the earlier plan, not a reduction in production standards.

2.2 Current implementation status

The greenfield implementation is now the supported application surface; the former legacy prototype has been retired:

- **Complete:** `backend/app` presentation, application ports, domain, configuration, AI fake, and persistence boundaries.
- **Complete:** deterministic complaint transition rules with idempotency, an accepted closure-proof claim boundary, and citizen-confirmation-gated closure. The real media/work-order verifier remains an injected production adapter rather than an invented local implementation.
- **Complete:** SQLAlchemy schema foundation for complaints, append-only events, sessions, and transactional outbox messages.
- **Complete:** Alembic configuration and PostgreSQL migrations through revision `0026`, including append-only database triggers for complaint events, department replies, silence events, evidence-review decisions, and citizen resolution responses, durable evidence/location metadata, complaint-to-evidence links, request fingerprints for complaint/upload/voice-draft idempotency without transcript-derived voice response persistence, minimal identity-verification records, encrypted short-lived authorization-state storage, notification delivery receipts, workflow-signal receipts, redacted closure-proof claims, persisted routing snapshot metadata, reviewed scheme/source records with reviewer identity, resumable evidence upload sessions/part receipts with cleanup claims, bounded escalation/disclosure-review state, one-time disclosure-consent metadata, opaque issue-cluster candidate/member records, private department-reply classification records, private deadline-breach facts with observed lifecycle status, immutable SLA timing snapshots, and durable media-review state. The complete chain has also been smoke-tested against PostgreSQL; revision identifiers are guarded to remain within Alembic's version-table limit.
- **Complete locally:** revision `0024_postgis_location_points` enables the PostGIS extension and adds a generated WGS84 geography point plus GiST index for captured locations. The local Compose database uses the PostGIS image, while live jurisdiction boundaries and operations contacts remain deferred; the routing port continues to use the bounded synthetic MP resolver until approved data is available.
- **Complete:** provider-neutral Agno adapter contract and structured-output tests using a fake agent; no live model calls are made.
- **Complete:** lazy Agno/Groq construction adapter with two tool-free, schema-constrained agents for intent and complaint extraction; construction makes no provider call and the API still requires explicit dependency injection.
- **Complete:** authenticated-principal boundary, DigiLocker sandbox verification adapter, evidence-verifier port, complaint creation command, citizen-scoped idempotency with unique-key recovery across database flush/commit races, and versioned complaint create/read/transition contracts.
- **Complete:** durable evidence/location metadata, direct object-storage upload-grant contract, a citizen- and idempotency-bound capture-attestation port, hash/size/content-type completion checks, and ownership/state verification for complaint submission. Production attestation remains an injected native/browser policy adapter.

- **Complete locally:** uncertain media inspection signals now enter durable `review_required` state instead of being silently accepted or rejected. An operator-only queue returns bounded metadata plus a short-lived provider-signed preview grant, and approve/reject decisions are capability-protected, idempotent, and append-only audited. The production malware/clarity/reuse inspector and reviewer policy remain activation gates.
- **Complete:** citizen-scoped complaint tracking with an explicit read contract and cross-citizen lookup protection.
- **Complete locally:** authenticated citizen tracking now includes a bounded timeline reconstructed from append-only lifecycle and silence events, plus the latest private citizen-resolution outcome. It exposes event type, safe status, reason code, escalation level, outcome, and time only; actor identifiers, raw event payloads, department reply text, and precise location remain excluded. Public receipt tracking remains the redacted point-in-time projection until public timeline policy is approved.
- **Complete locally:** policy-gated public accountability baseline at `GET /api/v1/public/transparency` and `/transparency`. It exposes only aggregate status, execution-zone, escalation, mapping-backlog, freshness, and policy-version facts; the flag is disabled by default and the projection contains no complaint IDs, citizen identity, raw text, contacts, precise locations, or evidence. Department response rates, duration percentiles, heatmaps, case pages, and exports remain separate policy/data-backed work.
- **Complete:** capability-gated lifecycle transition command with aggregate reconstruction, optimistic row locking where supported, idempotency, and event/outbox persistence.
- **Complete:** minimal identity-verification persistence and a transport-injected DigiLocker Requester policy adapter that allowlists claims and never stores raw documents.
- **Complete:** provider-neutral OIDC/JWKS bearer-token verification with issuer, audience, expiry, algorithm, and verified-identity claim checks.
- **Complete locally:** server-owned role-to-capability authorization policy in `backend/app/application/authorization.py`, with a versioned role matrix at `docs/security/CAPABILITY_MATRIX.md`. Protected admin, evidence-review, scheme-review, workflow-signal, and lifecycle-transition boundaries now use named capabilities rather than scattered endpoint role sets; unknown/viewer roles fail closed.
- **Complete:** bounded PostgreSQL outbox dispatch and SQS standard-queue publisher with explicit retry state, failure recording, and idempotent message envelope.
- **Complete:** deterministic Temporal complaint-lifecycle workflow, structured signals, replay-safe transition activity, worker factory, idempotent workflow starter adapter, and a persisted L1–L4 escalation cap with disclosure-review eligibility at L4; Temporal remains an orchestration layer over PostgreSQL/domain state.
- **Complete locally:** complaint intake resolves an explicit category-based SLA snapshot and persists its policy version, initial response timeout, and post-escalation timeout. The outbox carries the snapshot into Temporal, which never recomputes timing from mutable configuration. Synthetic category timings are the only current fixture; staging/production must inject an approved policy adapter.
- **Complete:** deployable outbox polling runtime, SQS event consumer with delete-on-success/redrive-on-failure behavior, and the `complaint.received` to Temporal workflow-start bridge. Duplicate starts use the durable complaint workflow ID as the effect-level deduplication key; completed duplicates recover the existing Temporal run handle instead of being retried indefinitely.
- **Complete:** provider-neutral DigiLocker authorization initiation/callback boundary with state hashing, PKCE S256, nonce, authenticated-subject binding, encrypted database state, atomic one-time consumption, and minimal callback response contracts.
- **Complete locally:** injectable DigiLocker Requester HTTP transport in `backend/app/infrastructure/digilocker.py`. It performs the configured authorization-code exchange with server-side Basic credentials, calls the configured authenticated user-details endpoint, applies an explicit scalar-claim allowlist, derives a non-secret consent reference, and never downloads or stores raw documents. The partner account, exact endpoint contract, approved claims, consent/retention policy, and deployment composition injection remain activation gates.
- **Complete:** opaque HMAC-backed complaint receipt tokens and a redacted public tracking projection; public reads cannot expose citizen identity, raw complaint text, jurisdiction precision, or location data.
- **Complete:** authenticated, non-authoritative complaint drafting at `POST /api/v1/complaints/draft`, using the provider-neutral `AgentOrchestrator` contract and Pydantic validation; the result cannot create, route, escalate, or transition a complaint.

- **Complete locally:** versioned server-owned complaint category catalogue at `GET /api/v1/complaints/categories`, with Hindi/English labels, pictograms, spoken Hindi prompts, and a bounded launch taxonomy. The mobile app consumes and securely caches the last valid catalogue for low-connectivity category selection; it does not invent category codes in the UI.
- **Complete:** authenticated voice-draft boundary at `POST /api/v1/complaints/voice-draft`. It accepts only a citizen-owned, server-verified audio asset, uses a narrow provider-neutral speech-to-text port, records provenance, passes only resulting text to the bounded extractor, and persists only the citizen-scoped request binding. A retry with the same key and request fingerprint re-runs the bounded flow so the transcript-derived read-back is never persisted. Missing speech adapters fail closed.
- **Complete:** bounded multi-handler conversation endpoint at `POST /api/v1/conversations/turn` with durable structured session context, citizen-scoped session ownership, turn idempotency, filing handoff, receipt-status guidance, and an explicit no-knowledge-base fallback for scheme questions. The router and extractor receive a compact context projection containing prior action and sanitized draft fields; its retry snapshot omits raw pre-submission complaint descriptions. It does not provide free-form scheme, legal, medical, political, or government-contact answers.
- **Complete locally:** the native citizen app now exposes that conversation boundary at `/chat` with low-literacy quick prompts, SecureStore-backed session continuity, authenticated access, stable idempotent chat retries, and explicit handoffs to DigiLocker verification, verified filing, receipt tracking, or source-cited scheme results. The client renders server-approved actions only; voice conversation input and signed-device acceptance remain provider/device validation gates.
- **Complete locally:** injectable deterministic Tone Governor at the conversation boundary. Casual turns containing political, medical, legal, financial-advice, or explicit threatening-language signals receive a neutral safe refusal; civic filing and reviewed scheme turns continue through their constrained handlers. It does not rewrite complaint facts or make domain decisions.
- **Complete locally:** provider-neutral department-reply ingestion at the existing operator workflow-signal endpoint. Replies are normalized, privately persisted, citizen/complaint-scoped, idempotent, and classified as substantive, weak, duplicate, or unavailable by a deterministic signal policy. Raw reply text never crosses the Temporal signal or lifecycle event payload; the outcome and closure-proof rules remain authoritative.
- **Complete locally:** Temporal timeout handling records an idempotent, append-only silence event before deterministic L1–L4 escalation. The event stores the workflow, observed lifecycle status, deadline, observed time, policy version, and escalation attempt without raw complaint/reply text; repeated L4 reminders remain individually queryable and appear as redacted breach entries in authenticated citizen timelines. No silence event is created when a structured response arrives before the deadline.
- **Complete:** greenfield Expo SDK 57 citizen capture/tracking flow under `apps/mobile`, with a one-time spoken Hindi/Hinglish first-use explainer, native camera-only capture, foreground GPS, mandatory voice note, voice-draft API handoff when no text is supplied, optional text fallback, spoken/tap read-back confirmation, an explicit private-by-default disclosure-consent step after filing, receipt-token tracking, a SecureStore-backed last-receipt tap path for non-reading users, icon/color/spoken status equivalents, and three authenticated citizen closure outcomes with stable idempotency keys (`fully_solved`, `partially_solved`, `not_solved`). The server persists each private outcome in the append-only `citizen_resolution_responses` boundary; only a fully solved outcome closes the case, while partial and unsolved outcomes reopen follow-up. The client preserves the SecureStore token boundary, stable per-capture idempotency keys, foreground retry on app reopen, and a SQLCipher-protected native queue whose key lives in SecureStore. Queued captures are bound to the local authentication session that created them, and retries skip mismatched sessions to prevent cross-account submission. It propagates `review_required` and rejected evidence states instead of treating them as successful uploads: uncertain captures remain retryable until operator approval, while terminally rejected captures are discarded from automatic retry. Live STT provider quality, dialect fallback, signed-build queue migration/at-rest verification, and real-device acceptance remain gated. The web build uses a deliberately non-durable queue fallback and is not treated as a verified-capture surface.
- **Complete locally:** separate Next.js App Router web surface under `apps/web` with accessible Hindi/Hinglish landing content, a public receipt-token tracking flow backed by the redacted API contract, a policy-gated aggregate transparency view at `/transparency`, optional authenticated citizen tracking with a bounded private timeline and the same three closure outcomes as native, icon/color status equivalents, browser-provided Hindi

speech playback when available, and an installable PWA shell with a narrowly scoped offline fallback. The public token view remains read-only and redacted; authenticated lookup is ownership-checked by the API, and the token is kept only in session storage during the OIDC return handoff. The service worker never caches API responses, receipt tokens, complaint data, or authenticated state; it explicitly bypasses `/admin` and its OIDC callback, and its public-shell cache is versioned when routes change. The `/file` route now provides DigiLocker-gated pictogram category selection, camera-only photo, browser GPS, microphone audio, audio-only voice-draft extraction with spoken read-back confirmation, direct upload grants, complaint submission when policy verifies both assets, and the explicit private-by-default disclosure choice before tracking is offered; browser evidence is separately labelled and review-gated by default, existing citizen-owned evidence IDs can be rechecked after operator approval, and photo/audio upload idempotency keys remain stable across retries until that capture is replaced.

- **Complete locally:** capability-protected admin control-tower reads at `GET /api/v1/admin/complaints` and `GET /api/v1/admin/overview`, plus the authenticated `/admin` web surface for aggregate workload, the redacted `mapping_in_progress` department-mapping queue, and bounded media-review decisions. The browser uses provider-neutral OIDC Authorization Code + PKCE with an in-memory access-token store; the API remains authoritative for roles/capabilities and returns no citizen identifiers, raw complaint text, or precise location. Production activation still requires the OIDC client, capability matrix, session/logout policy, CORS, and staging acceptance described in the action list.
- **Complete:** provider-neutral mobile OIDC authorization-code login with PKCE, SecureStore access-token storage, DigiLocker authorization handoff, citizen-scoped verification status, and a complaint-screen verification gate. The app does not contain a client secret, provider document claims, or invented DigiLocker endpoints. The current callback returns through the server and the app offers an explicit status refresh after the browser closes.
- **Complete:** server-side verification status projection and fallback authority. If an access token has not yet refreshed its verification claim, authenticated complaint/draft/evidence commands and conversation filing handoffs consult the persisted, citizen-scoped DigiLocker result; expired verification is not treated as verified.
- **Complete:** provider-neutral routing resolution in the complaint command plus a bounded synthetic MP adapter. It reads only server-owned evidence locations, activates only the documented synthetic Bhopal rectangle, and leaves missing, low-accuracy, and out-of-zone evidence in `mapping_in_progress`.
- **Complete:** durable notification-delivery boundary with channel/template contracts, opaque destination-reference hashing, request fingerprints, idempotency conflict detection, retryable failure state, provider receipts, and a deterministic fake sender. No real outbound provider or telecom channel is activated by this slice.
- **Complete:** privacy-safe HTTP observability boundary. It validates or generates `X-Request-ID`, returns it to clients, emits OpenTelemetry spans and bounded HTTP metrics through the pinned API/SDK, uses route templates instead of URL values, and logs only bounded method/status/timing/error metadata. Bodies, query values, tokens, complaint text, and provider error text are excluded; collector export, dashboards, alarms, and retention remain deployment actions.
- **Complete:** replaceable abuse-control boundary with hashed identity/IP/device dimensions, named AI/voice/evidence/complaint/identity/operator/public policies, atomic Redis-compatible adapter seam, deterministic test limiter, and bounded 429 responses. Staging/production construction rejects missing or process-local limiter adapters.
- **Complete:** staging and production application construction now reject any incomplete required `RuntimeAdapters` graph, not only a missing shared limiter or SLA policy. A complete deployment-owned bundle remains the only path to a deployed app; local/test entrypoints retain explicit test overrides. Provider clients, including the OIDC verifier, are constructed by that composition module rather than by the API entrypoint.
- **Complete locally:** AI and voice routes also enforce fleet-wide fixed-window request caps through the same limiter boundary (`AI_MONTHLY_REQUEST_LIMIT` and `VOICE_MONTHLY_REQUEST_LIMIT`). These are application safety guards, not currency budgets; staging and production require both values explicitly and fail closed when either is absent. Finance-owned provider billing alerts and approved values remain required before paid traffic.
- **Complete:** explicit liveness/readiness HTTP contracts. `/health` is provider-free process liveness; `/ready` performs a bounded database check and fails closed without a configured session factory or reachable database.
- **Complete:** durable, capability-protected Temporal signal commands for routing activation, department responses, and citizen confirmation at `/api/v1/admin/complaints/{complaint_id}/routing-activation`, `/`

`api/v1/admin/complaints/{complaint_id}/department-response`, and `/api/v1/complaints/{complaint_id}/citizen-confirmation`. Routing activation uses an injected server-owned resolver; the request contains no jurisdiction or contact fields. Each command has typed validation, role/ownership checks, lifecycle gating for citizen confirmation, a deterministic signal ID, a PostgreSQL receipt, retry-safe idempotency, and a provider-injected Temporal sender.

- **Complete:** department `fix_reported` signals now require an explicit typed proof reference. A provider-neutral verifier accepts or rejects the reference, the accepted claim is bound to the complaint and idempotency key, only its UUID/type/hash crosses the workflow boundary, and the domain transition rejects a fix report without an accepted claim. The default production verifier fails closed; the test fixture is explicitly isolated.
- **Complete:** grounded scheme-information boundary at `POST /api/v1/schemes/answer` and within conversation turns. It filters for approved scheme records, approved sources, requested language/jurisdiction, and current validity before returning a source-cited answer; it returns the safe no-answer response when those conditions are not met. The initial retrieval is deliberately simple structured lexical search; no model is allowed to invent eligibility facts.
- **Complete:** source-first scheme review boundary at `POST /api/v1/admin/schemes` and `POST /api/v1/admin/schemes/{scheme_id}/approve`. Structured records are staged as pending, HTTPS source metadata and retrieval dates are validated, natural-key retries are safe, and only a moderator/admin approval records reviewer identity and makes the scheme/source visible to public answers. No source catalogue is invented by this implementation.
- **Complete locally:** moderator/admin scheme review queue at `GET /api/v1/admin/schemes/review-queue` and the authenticated web review cards under `/admin`. The queue exposes bounded source, answer, eligibility, and version metadata; operators without `scheme.review` receive 403; approval removes the item from the queue and remains the only path to public grounded answers. No scheme catalogue is invented by this implementation.
- **Complete:** reproducible versioned contract artifacts in `docs/contracts/`: generated OpenAPI v1 for the HTTP surface and JSON Schema/Pydantic validation for the `complaint.lifecycle.v1` queue payload. Parity tests fail when the committed artifacts drift from FastAPI or the event model; AsyncAPI remains intentionally deferred until a broker-consumer catalogue exists.
- **Complete:** parameterized, disabled-by-default Terraform launch profile in `infra/terraform/` for ECS/Fargate API and worker services, private encrypted evidence storage, SQS/DLQ, CloudWatch logs, immutable image digests, non-root/read-only tasks, and runtime-scoped IAM/secret references. It validates locally without AWS credentials; applying it remains gated on approved network, KMS, secret, budget, image, and provider inputs.
- **Complete:** mobile capture retry now has a durable native SQLite queue, foreground retry, and an Expo SDK 57 `expo-background-task` registration with a module-scope task definition and stable idempotency keys. Successful foreground/background retries persist the server receipt token in secure storage for the last-receipt status path. It remains best-effort OS scheduling; a server receipt is still the only submission confirmation.
- **Complete:** evidence uploads now select single PUT for small assets and a provider-neutral multipart path for assets at or above 10 MiB. Multipart sessions persist the upload ID, 5 MiB part policy, presigned part grants, completed-part receipts, ETag/checksum metadata, retry-safe part completion, and final object verification; the S3 adapter passes recorded part SHA-256 checksums, explicit composite checksum type, and total object size to multipart completion. Presign failures map to the existing provider-unavailable boundary, and duplicate metadata races recover the persisted asset after rollback, enforce the request fingerprint, and regenerate grants for that asset instead of returning a grant for an uncommitted upload.
- **Complete:** explicit citizen disclosure consent at `POST /api/v1/complaints/{complaint_id}/disclosure-consent`. The command is authenticated, citizen-scoped, one-time, idempotent, append-only audited, and private-by-default; the server owns `PUBLIC_DISCLOSURE_POLICY_VERSION`, and `PUBLIC_DISCLOSURE_ENABLED=false` keeps public-name consent unavailable until product/legal policy is approved.
- **Complete locally:** a deterministic, non-destructive duplicate-candidate and same-area supporter-count boundary. It links related complaints without merging their lifecycle records, stores no raw location in cluster records, keeps citizen identity out of cluster membership through keyed references, and exposes no automatic escalation or rejection decision. Witness confirmation, abuse controls, and public collective views remain later policy-backed work.

- **Complete locally:** the implementation threat model and privacy review in [docs/security/THREAT_MODEL.md](#). It maps trust boundaries, restricted data, abuse cases, provider/deployment gates, and verification evidence. Worker failure logging was hardened at the same boundary to record only an error type, never exception text or traceback content that could contain citizen/provider data.
- **Complete locally:** the PostgreSQL backup/restore drill documented in [docs/operations/RECOVERY_DRILL.md](#). A PostgreSQL dump restored into an isolated database retained Alembic head `0026_voice_draft_request_binding` and all 24 public tables, including the PostGIS-managed `spatial_ref_sys` table and the append-only `citizen_resolution_responses` table/trigger. Production RDS backup, point-in-time recovery, restore access, RPO/RTO measurement, and derived read-model rebuild validation remain deployment actions.
- **Pending:** DigiLocker Requester partner onboarding, exact approved endpoint/claim mapping, consent/retention approval, and runtime composition injection; production native capture attestation, approved production media-inspector implementation and reviewer policy (the durable `review_required` queue and decision audit are implemented), an approved closure-proof verifier/media-proof policy, citizen disclosure/closure policy sign-off, approved STT provider/model, noisy-audio and dialect evaluation, human fallback for uncertain transcription, live Agno provider/model wiring and evaluation, approved scheme catalogue and semantic RAG evaluation, browser filing staging acceptance and the explicit `WEB_CAPTURE_REVIEW_REQUIRED=false` policy if direct browser verification is desired, production OIDC client and capability/session policy acceptance for the admin and citizen web surfaces, production routing-authority resolver and verified live jurisdiction/contact snapshots, approved category SLA policy adapter, signed-device validation, shared rate-limit deployment and WAF/edge policy, provider implementations supplied through the runtime composition module, approved network/KMS/secret/image wiring, and moderated citizen validation. The mobile lockfile is aligned to the latest compatible Expo SDK 57 patch set and `npm expo-doctor` passes; `npm audit --omit=dev` still reports 14 high and 7 moderate transitive Metro/Expo/React Native advisories with no compatible non-breaking fix, so signed-build security review and upstream remediation remain a release gate. The local roles/capabilities matrix and fail-closed checks are implemented; provider role registration, least-privilege assignments, audit review, and staging authorization tests remain activation gates. A non-root backend runtime image, worker command contract, and explicit `RuntimeAdapters` composition boundary are now implemented; external provider modules and approved production inputs remain deployment gates. A deterministic multilingual intent evaluation gate and source-review workflow are implemented; live Agno evaluation, approved catalogue content, and semantic retrieval remain gated by reviewed data/provider decisions. The abandoned multipart-upload cleanup worker is now implemented behind a database claim/abort/retry boundary; its S3 staging validation remains a deployment gate. The current slice is not being presented as the completed Phase 2 production vertical slice until these gates close.
- **Pending (priority policy):** the PRD names priority scoring, but the current implementation deliberately does not assign a priority because no approved non-commercial triage inputs, thresholds, safety policy, or audit contract exists. Product/domain owners must define bounded, explainable signals and confirm that identity, payment, political influence, and model output cannot affect priority, routing, or escalation. Until then, complaints are accepted without a priority score and the system does not invent one.
- **Deferred by decision:** live operations contacts/SLA data and telecom provider activation.
- **Pending (observability deployment):** provision an in-network OpenTelemetry Collector/OTLP endpoint, configure CloudWatch or the selected telemetry backend, define dashboard/alert ownership and retention, and verify that telemetry exports contain only the bounded dimensions implemented by the API.

The implementation evidence and local commands are documented in the [greenfield backend README](#), the [runtime boundary ADR](#), and the [threat model/privacy review](#). The current endpoints accept only server-derived identity and server-owned verified evidence asset IDs; they do not treat client-supplied citizen IDs, file references, or sandbox verification as production authentication.

3. Product boundary

In scope

- Citizen complaint filing, verification, routing, tracking, reminders, escalation, and closure.
- Voice-first interaction in Hindi, English, Hinglish, and progressively supported regional dialects.
- Web, native mobile, WhatsApp, IVR, SMS/USSD, and assisted/operator intake adapters.
- Government-scheme information grounded only in an approved, versioned knowledge base.
- Public transparency, case pages, neutral reports, and evidence export.
- Role-based operations and auditability; live district/contact configuration is a future activation boundary, while the current slice uses synthetic routing fixtures.

Explicitly out of scope

- Paid complaint priority.
 - Political endorsement, politician tagging, election conversation, or campaign tooling.
 - Officer naming/shaming, accusatory language, legal threats, or fabricated statutory references.
 - General-purpose medical, legal, or financial advice.
 - Unverified scheme eligibility claims.
 - Family or village-representative filing on behalf of another person without an explicit supported identity/consent flow.
-

4. Architecture shape

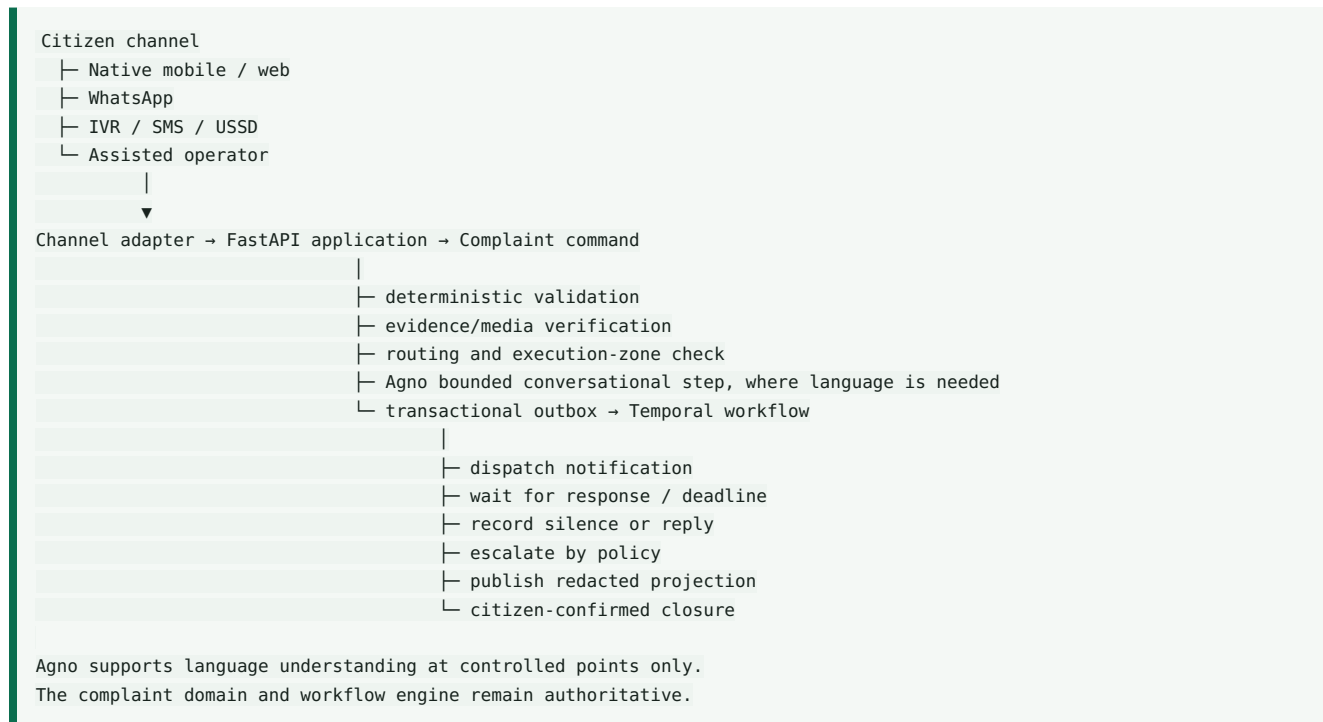
4.1 Recommended operating model

Use a **modular, event-driven platform** with these initial deployable units. Several are modules in one Python deployment at first; they become separate workers or services only when a measured reliability, security, scale, or ownership boundary requires it.

Deployable unit	Responsibility	Must not own
FastAPI application	Authentication, request validation, channel normalization, rate limits, public API, application orchestration	Provider-specific SDK calls or hidden complaint decisions in route handlers
Complaint domain modules	Complaint aggregate, evidence metadata, validation rules, lifecycle transitions, routing, policy evaluation	LLM calls, email/WhatsApp SDKs, or direct workflow timers
Agno AI module	Intent routing, bounded specialist handlers, structured extraction, grounded response rendering, AI safety checks	Direct status, routing, priority, escalation, disclosure, or closure writes
Temporal worker	Durable timers, SLA clocks, reminders, escalation commands, closure timers, retry policies	Free-form language interpretation or owning domain facts
Background workers	Media inspection, communications, scheme ingestion, projections, exports, and provider adapters	Deciding whether a domain transition is warranted
Scheme knowledge module	Approved scheme data, source documents, versioning, retrieval, citations	Answers based on general model memory
Web application	Citizen web, public transparency, admin/control tower	Holding secrets or authoritative workflow state
Native mobile application	Camera, voice, location, offline queue, accessible interaction	Bypassing server-side verification

At the beginning, these units live in one repository and can share a small number of runtime images. They communicate through application ports, a transactional outbox, and versioned events. Split deployment only when load, reliability, security, or team ownership justifies it.

4.2 Request and event flow



4.3 Source-of-truth rules

- PostgreSQL is the transactional source of truth for current domain state.
 - Append-only domain events are the evidence trail for every meaningful transition.
 - A transactional outbox publishes those events reliably after the domain commit; it is a delivery mechanism, not a second source of truth.
 - Workflow history is retained as operational evidence and linked to domain events.
 - Object storage is the source of truth for original media; the database stores metadata, hashes, classification, retention, and access policy.
 - Read models are disposable projections and can be rebuilt from events.
 - No browser local storage, process memory, mutable JSON file, or LLM conversation history may be the only copy of user-visible workflow state.
-

5. Technology stack decision

5.1 Primary stack

Layer	Recommendation	Reason
Repository	Polyglot monorepo: Python backend plus TypeScript web/mobile clients	Keeps Agno and the API in one runtime while preserving shared client contracts and UI code
Public/admin web	Next.js App Router, TypeScript, accessible component system	Strong server/client separation, production build, public pages, dashboards, and PWA fallback
Native citizen app	React Native with Expo, Expo Router, native builds through EAS	Shared TypeScript skills with web; camera, video, location, audio, secure storage, SQLite, and offline queues
API/application	FastAPI with Pydantic, SQLAlchemy, and Alembic	Matches the existing Python boundary, integrates Agno in-process, and provides typed validation and OpenAPI without a second application runtime
API contract	REST with OpenAPI 3.1/3.2-compatible generated contracts	Simple integration for web, mobile, WhatsApp, IVR, operators, and government consumers
Event contract	JSON Schema plus AsyncAPI when broker consumers are introduced	Versioned, reviewable domain events without requiring a broker for the first vertical slice
Durable workflows	Temporal with the Python SDK	Crash recovery, durable timers, retries, signals, and long-running complaint workflows in the same backend language
Event delivery	Transactional PostgreSQL outbox plus Amazon SQS workers at launch; Kafka-compatible broker only at measured scale	Managed, low-operations background delivery with explicit at-least-once/idempotent handling and a clean path to replayable topics later
Transactional database	PostgreSQL with PostGIS, read replicas, and declarative partitioning	Strong consistency, spatial verification, jurisdiction lookup, reporting, and mature SQL
Scheme retrieval	PostgreSQL full-text search plus pgvector initially; dedicated search only when measured scale requires it	Keeps authoritative scheme facts, versioning, filters, and vector retrieval close together
Media	S3-compatible object storage with CDN and short-lived presigned URLs	Durable, scalable media storage without passing large files through the API
Identity	OIDC-compatible account layer plus DigiLocker Requester integration for citizen verification	Consent-based government document verification, minimal stored identity data, and a replaceable integration boundary
AI orchestration	Agno in a bounded Python AI module, behind an <code>AgentOrchestrator</code> interface	Lowest integration friction with FastAPI; agents, workflows, structured Pydantic output, sessions, guardrails, and tracing are available without a second AI runtime
Speech/language	Bhashini and approved commercial/self-hosted STT/TTS adapters; model/version recorded for each output	Indian-language coverage, dialect experimentation, fallback providers, and cost control
Observability	OpenTelemetry plus CloudWatch metrics/logs/alarms at launch; Prometheus/Grafana/Loki/OpenSearch when operational volume justifies them	Useful production monitoring with low initial operations cost and a portable telemetry boundary
Infrastructure	AWS India regions: ECS/Fargate, RDS PostgreSQL, S3, CloudFront, KMS, Secrets Manager, WAF, and SQS; EKS/Kafka only after measured scale requires them	Production-capable managed primitives without paying the operational cost of Kubernetes and Kafka prematurely
Infrastructure as code	Terraform or OpenTofu with reviewed modules	Repeatable environments and disaster-recovery reconstruction
Delivery	GitHub Actions, container scanning, signed images, and a reviewed ECS promotion path; GitOps when Kubernetes is adopted	Reproducible promotion without requiring a Kubernetes control plane
Testing	pytest, Playwright, OpenAPI contract tests, k6 load tests, and Agno/model evaluation fixtures	Domain, browser, contract, load, and AI quality coverage in the selected runtimes

The backend pins `opentelemetry-api==1.44.0`, `opentelemetry-sdk==1.44.0`, and `opentelemetry-exporter-otlp-proto-http==1.44.0`. The application uses manual, low-cardinality instrumentation and sends only to a deployment-owned OpenTelemetry Collector; local/test runs remain no-op unless a deterministic exporter is injected. This follows the official [Python instrumentation guidance](#) and [OTLP exporter guidance](#).

The production version of Next.js should be selected from the current stable release at project bootstrap and pinned in the lockfile. The current Next.js documentation recommends the App Router for new work, documents production builds, PWA guidance, OpenTelemetry integration, and testing options. See the [official App Router documentation](#), [installation and production build guidance](#), and [production guides](#).

FastAPI provides typed request/response validation and generated OpenAPI documentation; Pydantic, SQLAlchemy, and Alembic keep contracts, persistence, and migrations explicit. Verify and pin the versions at bootstrap using the [FastAPI documentation](#), [Pydantic documentation](#), [SQLAlchemy documentation](#), and [Alembic documentation](#).

Agno is selected for the multi-agent layer because its documented primitives cover agents, teams, workflows, structured Pydantic output, database-backed sessions, and guardrails. Use Agno's workflows for bounded conversational orchestration; keep complaint lifecycle truth outside it. See the [Agno agent overview](#), [workflow patterns](#), [session storage](#), and [guardrails](#).

Temporal is chosen for complaint lifecycle workflows because it is designed to resume long-running work after crashes, network failures, and infrastructure outages. Use the Python SDK in the backend boundary and start with Temporal Cloud for the pilot, subject to India-region/data-processing and cost review; self-hosting is a later option if sustained volume makes it economically preferable. See the [Temporal documentation](#), [Python SDK guidance](#), and [Temporal Cloud cost guidance](#).

PostGIS is chosen because it extends PostgreSQL with spatial objects, indexes, and analysis functions. See [PostGIS getting started](#) and the [PostGIS manual](#).

Expo provides the native camera, video, file-system, SQLite, secure-storage, and best-effort background-task building blocks required by the low-literacy/offline strategy. See [Expo Camera](#), [Expo SQLite](#), [Expo BackgroundTask for SDK 57](#), and [Expo data storage guidance](#).

AWS ECS/Fargate, RDS for PostgreSQL, S3, SQS, read replicas, partitioning, presigned URLs, and S3 multipart upload provide the cost-controlled reference production primitives. See [ECS/Fargate](#), [SQS standard queues](#), [RDS for PostgreSQL](#), [RDS read replicas](#), [RDS partitioning guidance](#), [S3 presigned URLs](#), and [S3 multipart upload](#).

5.2 Why this is not “microservices everywhere”

The mass-scale research correctly identifies the need for independent scaling and event-driven processing. However, independently deploying every feature from the first commit would increase operational risk and slow delivery. The chosen approach is:

- one polyglot monorepo with a Python application boundary and TypeScript clients;
- clear presentation, application, domain, infrastructure, and operations modules;
- one API runtime, one workflow worker, and a small number of background worker processes initially;
- durable workflows for time-based business processes;
- versioned events and a transactional outbox from the first vertical slice;
- a broker and separate services only when measured load, reliability, or ownership requires them.

This preserves the scale path without creating a distributed system whose behavior is impossible to test locally or reason about.

5.3 Cost-controlled launch profile

The budget constraint changes the launch profile, not the domain guarantees:

- Run the API, Agno module, and workers as right-sized ECS/Fargate services with autoscaling and scale-to-zero for non-production environments.

- Use RDS PostgreSQL with Multi-AZ for production data, point-in-time recovery, and no read replica until measured read load requires one. Single-AZ is acceptable only for local/development environments.
- Use S3 for media and SQS for background work; consumers must be idempotent because standard queues provide at-least-once delivery and may redeliver or reorder messages. See the [SQS delivery documentation](#).
- Use Temporal Cloud for the pilot with consumption dashboards, bounded retries, sensible backoff, and workflow history retention controls. Do not poll aggressively or create a workflow per conversational turn.
- Start with approved managed AI/STT/TTS providers behind adapters, strict per-provider quotas, prompt/token budgets, and cost-per-complaint metrics. Evaluate self-hosted inference only after measured volume and GPU economics justify it.
- Keep application-level AI/voice request caps in the shared limiter (`AI_MONTHLY_REQUEST_LIMIT` and `VOICE_MONTHLY_REQUEST_LIMIT`) as a last-resort fleet-wide spend guard. Pair them with provider currency alerts and quota controls; the application must not pretend to calculate spend without the selected provider's current pricing.
- Use CloudWatch-backed alerts initially; add a larger Prometheus/Grafana/Loki/OpenSearch footprint when the number of services and operators justifies the cost.
- Do not introduce EKS, Kafka, read replicas, multi-region active/active deployment, or dedicated GPU clusters until an evidence-based threshold is reached and documented.

This keeps the first production-capable environment small, observable, recoverable, and replaceable while avoiding infrastructure that would consume budget before citizen usage validates the product.

5.4 Deliberate non-choices

- No flat “do everything” agent.
- No use of an agent framework merely to implement a deterministic function or status lookup.
- No LangGraph.js, OpenAI Agents SDK, CrewAI, or second agent framework in the baseline; Agno is the selected orchestration boundary.
- No LLM as the complaint state machine or escalation authority.
- No raw media in PostgreSQL.
- No mutable JSON backup as a business data store.
- No global browser-exposed admin secret.
- No direct provider SDK imports in domain/application rules.
- No Kafka consumer used as a substitute for durable workflow state.
- No raw identity document storage by default.
- No exact citizen coordinates in public projections.
- No personalized avatar video per citizen; avatar videos are periodic public summaries only.

5.5 Framework selection record

Option	Decision	Reason
Agno SDK	Selected	Python-native, fits FastAPI in-process, supports bounded agents/workflows, structured Pydantic output, database-backed sessions, and guardrails; lowest integration cost for this repository and team direction
LangGraph.js	Not selected for the baseline	Strong graph persistence and human-in-the-loop primitives, but it would introduce a separate JavaScript AI runtime while the application and model/provider adapters are Python-oriented; reconsider only if graph-specific needs outweigh that cost. See the LangGraph persistence documentation .
OpenAI Agents SDK	Not selected for the baseline	Useful tools, handoffs, guardrails, and tracing, but it would create a second orchestration choice without solving the durable complaint workflow; use only if a later provider/platform decision makes it materially simpler. See the official Agents SDK documentation .
Temporal	Selected alongside Agno	Not an AI framework; it owns durable, replay-safe complaint/SLA execution, timers, retries, signals, and escalation.

The decision is based on total system integration cost, not on which framework has the largest feature list. Agno remains behind a replaceable port. The repository has now pinned and locally validated the baseline `agno==2.6.5` and `groq==1.6.0`; production use still requires the legal, provider-terms, persistence, and staging evaluation checks listed below.

6. Multi-agent and AI orchestration design

The user experiences one AI Neta. Internally, the system uses narrow handlers with explicit permissions.

Agno is the orchestration framework for these bounded interactions. Use an Agno `Router` or deterministic Python selector for intent, Agno `Agent` instances for narrow language tasks, and Agno `Workflow` steps for a known sequence. Use `Team` only for non-critical collaborative tasks such as research or content preparation; do not use a dynamically delegating team for complaint submission or escalation.

The `AgentOrchestrator` interface must keep Agno out of domain code. This makes the framework replaceable and lets tests use deterministic fakes without model calls.

6.1 Handler boundaries

Handler	Allowed responsibility	Forbidden responsibility
Intent router	Classify the next turn as casual, scheme, filing, status, or continuation	Mutating complaints or inventing missing facts
Complaint extraction	Convert citizen language into a structured draft with confidence and missing fields	Approving evidence, assigning escalation, or deciding closure
Complaint conversation renderer	Generate short prompts/read-backs from approved workflow instructions	Adding facts not present in the workflow context
Scheme information handler	Answer only from retrieved, current, cited scheme records	General-knowledge eligibility guesses
Casual chat handler	Safe everyday conversation and guided handoff to filing	Political, medical, legal, or ungrounded government claims
Status renderer	Convert a database-backed status projection into text/audio/icon output	Explaining unknown events or making predictions
Tone governor	Detect abuse, threats, accusatory wording, and unsafe outbound language; suggest neutral rewrites	Changing facts, SLA outcomes, or evidence conclusions
Evidence classifier	Produce media quality/reuse/context signals	Making a final legal or identity determination

6.2 Shared context

All handlers receive a compact, persistent session context containing:

- `session_id` and authenticated citizen identity reference;
- channel and device capability;
- selected language/dialect and voice preferences;
- current workflow state;
- complaint draft ID, if any;
- verified fields already collected;
- approved facts allowed for response;
- previous response IDs and event IDs;
- consent and disclosure state;
- handoff reason and next workflow command.

Handlers do not pass free-form private transcripts to each other as the contract. They pass structured context and reference IDs. The persona layer renders the final answer in the consistent AI Neta voice.

6.3 AI gateway controls

Every model call must record:

- provider and model ID;
- prompt/policy version;
- input classification;
- retrieved source IDs, if applicable;
- structured output validation result;
- safety result;
- latency, token/cost estimate, and retry count;
- final response ID.

The gateway must support provider fallback, timeout, circuit breaking, redaction before logging, and deterministic fallback prompts. Model output must be validated against JSON Schema before application code can use it.

Agno's Pydantic output validation is useful at the AI boundary, but it is not a business authorization check. The application must validate the resulting command again against the current complaint aggregate, authenticated actor, policy version, and idempotency key.

The initial integration uses the Agno SDK inside the existing FastAPI application boundary, not a second public AgentOS API. This avoids duplicate authentication, tenant/session boundaries, and PII paths. AgentOS may be evaluated later for an internal agent operations/evaluation surface after security, data-retention, and licensing review. Agno session/history storage is operational context only; the application-owned `sessions`, complaint drafts, events, and audit records remain authoritative. Production persistence must be managed through the project's migration and retention process rather than relying on startup auto-created tables.

6.4 AI signal versus business decision

The PRD's "AI Decision Core" is implemented as a two-step boundary:

PRD capability	AI may produce	Deterministic authority
Complaint approve/reject	Category, missing fields, evidence-quality signals, duplicate candidates	Verification policy and complaint domain command
Priority scoring	Explainable features and a bounded score suggestion	Versioned priority policy and threshold rules
Escalation approval	Weak-reply or delay signals	SLA workflow and escalation ladder
Weak-reply detection	Classification with reason codes	Review policy and escalation command
Public disclosure approval	Redaction suggestions and policy checks	Citizen consent and disclosure policy
Closure	Proof-quality signals	Proof policy plus citizen confirmation

No model output is sufficient by itself to mutate critical domain state.

6.5 Scheme/RAG safety

The scheme service stores structured facts, source URLs/documents, jurisdiction, language, eligibility dimensions, effective dates, expiry dates, and review status. Retrieval must filter by location, language, audience, and validity before similarity ranking.

If no reviewed source supports an answer, the response must say that verified information is unavailable and provide a safe next action. The model must never fill gaps from general knowledge.

7. Core domain model

The greenfield schema should be designed around aggregates and immutable events rather than a single wide complaint row.

7.1 Core entities

- `citizens`: minimal account identity and channel links.
- `identity_verifications`: provider, method, assertion reference, consent, outcome, and retention policy; no raw document by default.
- `sessions`: persistent cross-channel conversation/workflow context.
- `complaints`: current aggregate state, jurisdiction, category, priority, disclosure policy, execution-zone state, and version.

- `complaint_drafts`: pre-submission fields and user confirmation state.
- `evidence_assets`: object key, type, hash, capture metadata, verification signals, retention, and access policy.
- `location_samples`: coordinate, accuracy, timestamp, source, jurisdiction result, and confidence.
- `routing_cases`: department/contact snapshot, active zone, verification status, and activation time.
- `sla_policies`: category, urgency, jurisdiction, working-calendar, response deadline, resolution target, and escalation thresholds.
- `workflow_instances`: workflow ID, version, state, and links to Temporal execution.
- `department_replies`: inbound channel, raw provider reference, normalized response, received time, and classification.
- `escalation_attempts`: level, target snapshot, reason, scheduled/sent/acknowledged times, delivery evidence, and result.
- `closure_claims`: department proof and citizen outcome request.
- `citizen_resolution_responses`: fully solved, partially solved, not solved, and reopen reason.
- `notifications`: channel, template/version, destination reference, idempotency key, delivery status, and provider receipt.
- `witness_confirmations`: one-time confirmation token, locality boundary, abuse controls, and privacy policy.
- `issue_clusters`: opaque, policy-derived location-cell/category/time-window candidate and distinct supporter count; raw coordinates are not copied into the cluster record.
- `audit_events`: append-only actor/action/resource/before-after hash/correlation ID.
- `scheme_records` and `scheme_sources`: versioned, reviewed, cited government information.
- `roles`, `capabilities`, `assignments`, and `district_configurations`: authorization and operating configuration.

7.2 Public status projection

Internal states may be detailed, but public status should remain understandable:

```
Received → Verifying → Routed → Sent → Awaiting response
    → Escalated → Fix reported → Citizen confirmation
    → Closed

Missing verified contact → Mapping in progress
Citizen says not solved → Reopened
Validation failure → Needs clarification / Not accepted
```

The public projection must not expose internal fraud scores, identity data, private notes, raw officer contact information, or precise coordinates unless explicitly approved by policy.

7.3 State transition rules

- Every transition is a domain command validated against the current version.
- Transitions are idempotent and carry an idempotency key.
- The API returns the current version and rejects stale writes.
- Citizen-submitted facts cannot be silently overwritten; corrections are new events.
- Status changes cannot delete history.
- A department reply is evidence of communication, not proof of resolution.
- A case closes only after accepted proof plus citizen confirmation.
- “Not solved” creates a reopen event and starts the next workflow segment.

8. Verified complaint workflow

8.1 Native mobile path

1. Explain the process with a short local-language audio/video primer on first use.
2. Start an authenticated or risk-scored session.
3. Capture photo/video only through the native camera component; no gallery picker in the verified evidence step.
4. Capture GPS, accuracy, device time, server receipt time, and capture source.
5. Record the mandatory voice note for infrastructure categories.
6. Compress media on-device while preserving the original hash and capture metadata.
7. Upload directly to object storage using a short-lived presigned URL.
8. Run server-side MIME, size, malware, clarity, duplicate/reuse, and provenance checks.
9. Resolve the jurisdiction using PostGIS and the approved boundary dataset.
10. Use the extraction handler to produce category, description, language, and missing-field candidates.
11. Apply deterministic validation rules.
12. Read back the complaint in the citizen's language and show icon/voice confirmation.
13. Ask for disclosure choice and consent.
14. Submit once with an idempotency key.
15. Generate the receipt immediately, even if routing or notification is asynchronous.
16. Start the durable complaint workflow.

8.2 Web path

The web experience should support the same business contract but use the strongest available browser capabilities. Verified evidence should either:

- open a camera-only capture component with an explicit no-gallery policy and server checks; or
- classify the complaint as lower-verification until the native app, assisted operator, or another approved capture path completes the required evidence.

The PWA is an access channel, not an excuse to weaken the evidence policy.

8.3 WhatsApp and IVR paths

Channel adapters normalize media, audio, location, message IDs, and consent into the same commands used by the app. They do not implement a second complaint domain.

- WhatsApp uses the official Business/Cloud API with signed webhook verification, message deduplication, template governance, and delivery receipts.
 - IVR supports voice prompts, keypad fallback, callback scheduling, complaint status, and receipt delivery.
 - SMS/USSD provides low-bandwidth status and receipt fallback where legally and operationally available.
 - Assisted operators use the same APIs with a distinct actor role and immutable audit trail.
-

9. Verification, privacy, and anti-abuse

9.1 Media verification

- Native capture token tied to session and device installation.
- Original object hash plus normalized derivative hash.
- Capture timestamp and server receipt timestamp.

- EXIF and container inspection, with no assumption that EXIF is trustworthy.
- Multi-frame/video sampling for continuity and screenshot/reuse signals.
- Perceptual hash and prior-evidence matching.
- Malware and decompression-bomb protection.
- Human review queue for uncertain cases.
- A verification result with reason codes, not a binary “AI says real” decision.

No system can mathematically guarantee that a camera scene is truthful. The platform should enforce capture provenance and transparently record verification confidence and limitations.

9.2 Location verification

- GPS accuracy and age thresholds.
- Network/provider location as a separately labeled source.
- Cross-check between selected jurisdiction, GPS, and submitted description.
- PostGIS boundary lookup and nearest-area calculation.
- Suspicious mock-location/device-integrity signals where available.
- Public geospatial generalization using H3/geohash cells or ward-level aggregation.

9.3 Identity and consent

- OIDC identity for sessions and roles.
- Phone OTP for low-friction account access, protected by rate limits and abuse detection.
- DigiLocker Requester integration as the primary identity-verification adapter, using its official OAuth/OIDC and consent flow.
- Store only the DigiLocker verification/reference ID, verified document type or claims required by policy, outcome, consent receipt, timestamps, and retention deadline unless a lawful requirement says otherwise.
- Do not store the full DigiLocker document by default. Retrieve only the minimum claims needed for the approved verification purpose, and record the access audit trail.
- Separate “verified citizen” from public name disclosure.
- Public disclosure is a one-time, versioned consent decision with clear audio and visual explanation.

DigiLocker requires requester registration, OAuth/OIDC credentials, user consent, and controlled access to documents. This remains an onboarding and compliance task rather than an assumed API key. See the [DigiLocker Requester integration guidance](#), [DigiLocker security architecture](#), and [API Setu specifications](#). DPDP purpose limitation, retention, and access controls still apply.

9.4 Anti-abuse

- Per-identity, per-device, per-IP, per-channel, and per-jurisdiction rate limits.
- Duplicate and coordinated-submission detection.
- Evidence reuse detection.
- Progressive trust levels rather than paid priority.
- CAPTCHA/step-up controls only where they do not exclude low-literacy users.
- Human moderation for uncertain or high-impact clusters.
- Appeal and correction process for rejected or merged complaints.
- No public display of abuse scores or citizen identity.

The API boundary hashes identity, direct-connection IP, and optional device signals before sending them to the rate-limit store. It never trusts a device signal for authentication or authorization, and it does not accept arbitrary forwarded-IP headers without a separately approved proxy trust policy. The production store must be shared across API instances; the in-memory fixture is limited to tests.

10. SLA, silence tracking, and escalation

10.1 Workflow model

Each active complaint starts a versioned workflow containing:

- policy version;
- category and urgency;
- jurisdiction and active-zone status;
- routing/contact snapshot;
- response deadline;
- resolution target;
- working calendar and holidays;
- current escalation level;
- pending notification or reminder activities;
- pause/resume rules, if any;
- closure and reopen policy.

Temporal owns timers and retries. PostgreSQL and append-only events own business facts. The workflow must be deterministic and replay-safe.

10.2 Silence tracking

Record every meaningful time point:

- accepted;
- routed;
- notification attempted;
- notification delivered;
- department acknowledged;
- reply received;
- reply classified as substantive/weak/duplicate;
- deadline approaching;
- deadline breached;
- escalation scheduled;
- escalation sent;
- citizen follow-up sent;
- proof submitted;
- citizen confirmation requested;
- citizen response received.

The public transparency model can expose aggregated breach and delay statistics without exposing private complaint text or exact personal locations.

10.3 Escalation ladder

The ladder is configuration data, not model output:

- **L1:** concerned local officer.
- **L2:** senior/zonal officer.
- **L3:** department head.
- **L4:** district/state authority.

Escalation occurs only when the configured rule is met. Weak replies may trigger a senior copy, but the classification must be explainable, versioned, and reviewable.

10.4 Execution-zone behavior

Condition	Citizen-facing state	Outbound behavior
Verified active zone and complete routing	Sent / Awaiting response	Email, WhatsApp, SMS, or IVR according to consent and policy
No verified contact	Mapping in progress	No blind outbound dispatch
Contact awaiting verification	Mapping in progress	Admin queue and audit event
Contact revoked or delivery repeatedly fails	Routing review	Pause and create an operational alert

11. Closure and proof model

Department proof

Accepted proof types are category-configurable and may include:

- after photo/video captured through an approved path;
- work order or service reference;
- location/time metadata;
- department response;
- citizen or witness confirmation;
- human review for ambiguous cases.

The implementation boundary is provider-neutral: a department response must submit a typed proof reference, the configured verifier must accept it, and the API persists only a complaint-bound claim ID, proof type, verifier, timestamps, and a one-way hash of the reference. The raw reference is not placed in the workflow signal, event payload, logs, or public projection. The current local fixture is test-only; production requires an approved media/work-order/human review verifier and retention policy.

Citizen outcome

The citizen receives a voice-first request with three choices:

- Fully solved.
- Partially solved.
- Not solved.

“Partially solved” keeps the case open with a reduced or revised target. “Not solved” reopens the workflow. Department self-report alone never closes the complaint.

Temporary fixes

The policy engine must distinguish temporary mitigation from resolution. For example, a one-time garbage pickup or temporary patch may generate a follow-up task rather than a closed state.

12. Low-literacy and first-time-user experience

The primary path must be completable without reading.

- Spoken prompts by default, not behind an accessibility setting.
- Tap-first pictograms for the server-owned launch categories, with spoken labels and one action per screen. Additional categories require a catalogue/policy update rather than a UI-only change.
- Short steps with one action per screen.
- Voice read-back before submission.
- Color/icon equivalents for each public status.
- Audio status via WhatsApp voice message or IVR callback.
- Local-language onboarding explaining what happens after filing.
- Dialect-tolerant speech recognition and human fallback for uncertain audio.
- SMS/printed/partner receipt with the complaint ID.
- Offline queue with clear “saved on this phone / not yet submitted” state.
- Compression and resumable upload for poor networks.
- No requirement to type a long address.
- No requirement to understand department hierarchy.

The native mobile app should use `expo-camera`, `expo-location`, `expo-audio`, `expo-file-system`, `expo-sqlite`, and secure storage. The web/PWA is an enhancement and fallback, not the only trustworthy capture surface; its browser evidence remains review-gated unless an explicit policy approves direct verification.

13. Public transparency and admin control tower

Public experience

- Aggregated department response rate.
- Median and percentile pending duration.
- Area heatmaps at privacy-safe resolution.
- Status and escalation distribution.
- Public case pages with neutral language and consent-filtered evidence.
- Downloadable case bundle where disclosure permits it.
- Weekly/monthly reports generated from read models.

The first implementation slice is deliberately narrower: a disabled-by-default aggregate status/zone/escalation projection is available for policy review. It does not imply that live department or operations data is present, and it does not enable case-level publication.

Admin experience

- Complaint queue with state, SLA clock, escalation level, zone, and verification reason.
- Contact-mapping review queue with verification and revocation workflow; live hierarchy/contact data remains deferred.
- Contact verification and revocation workflow.
- Repeated-issue and geospatial cluster view.
- Delivery failures and notification provider health.
- Review queues for media, identity, weak replies, and citizen appeals.
- Role-specific dashboards for Admin, Moderator, Viewer, and Volunteer.
- Configuration changes require approval, versioning, and audit events.

- No raw citizen PII in dashboards unless the role and case purpose allow it.

Evidence exports

Generate a deterministic case bundle containing:

- complaint ID and public metadata;
- event timeline;
- routing/contact snapshots;
- SLA policy version;
- notification and delivery evidence;
- department replies;
- escalation history;
- evidence references and hashes;
- citizen consent and closure response;
- export timestamp and bundle hash.

PDF generation should be a worker job from an immutable export request, not a browser-generated screenshot.

14. Repository and package layout

```
apps/
  web/          # Next.js public, citizen web, admin, transparency
  mobile/       # Expo React Native citizen app

backend/
  app/
    api/        # FastAPI routes, auth dependencies, request/response contracts
    application/ # Use cases, command/query orchestration, idempotency
    domain/     # Aggregates, policies, state transitions, domain events
    infrastructure/ # SQLAlchemy repositories, storage, messaging, provider adapters
    ai/         # Agno agents/workflows behind AgentOrchestrator ports
    knowledge/   # Scheme ingestion, review, retrieval
    workflows/  # Temporal workflow definitions, activities, and worker factory
    workers/    # Media, communications, projections, exports, and runtime entrypoints
    migrations/ # Alembic migrations
    tests/      # Unit, integration, contract, workflow, and AI evaluation tests

packages/
  contracts/    # OpenAPI, JSON Schema, AsyncAPI when broker consumers exist
  ui/           # Accessible shared UI primitives
  i18n/         # Translations, audio prompts, dialect metadata
  test-fixtures/ # Deterministic data and provider fakes

infra/
  terraform/    # AWS/network/data/platform infrastructure
  kubernetes/   # Added when the production runtime uses Kubernetes
  observability/ # OTel collectors, dashboards, alerts

docs/
  architecture/ # ADRs, sequence diagrams, threat models
  operations/   # Runbooks, incident response, DR exercises
  product/      # Policy, accessibility, consent, SLA definitions
```

Each deployable runtime must have a README, configuration schema, health/readiness behavior, API/event contract, test strategy, and owner. Domain and application modules may not import provider SDKs directly. No worker may bypass domain commands to mutate complaint state.

15. Delivery roadmap

The schedule below is indicative. Each phase ends with a usable, testable vertical slice and explicit exit criteria.

Phase 0 — Decisions, policy, and platform bootstrap

Objective: remove ambiguity before building irreversible workflows.

Deliverables:

- Approved product boundary and public-disclosure policy.
- Category taxonomy and synthetic MP SLA fixtures.
- Active-zone and contact-verification schema, with live operations data intentionally deferred.
- DigiLocker integration/legal decision record.
- Data classification and retention matrix.
- Threat model and abuse model.
- Polyglot monorepo, Python/TypeScript tooling, CI, coding standards, contract generation, and ADR template.
- AWS India-region reference account/network design.

Exit criteria:

- Product, legal, and engineering owners sign the decisions required for the build slice.
- Synthetic fixtures cover routing, SLA, contact verification, and mapping-in-progress behavior.
- No live outbound automation is enabled without the later operations-data gate.

Phase 1 — Domain and platform foundation

Objective: build the durable backbone before conversational features.

Deliverables:

- OIDC identity and capability-based authorization.
- Durable DigiLocker authorization state with encrypted PKCE material and one-time callback consumption.
- PostgreSQL/PostGIS schema and migration pipeline.
- Complaint aggregate, domain events, audit events, idempotency, and optimistic concurrency.
- Object-storage upload flow with metadata/hash records.
- Deterministic Temporal complaint workflow boundary, replay-safe transition activity, worker factory, idempotent starter adapter, transactional outbox, and deployable SQS/Temporal worker runtimes.
- Generated OpenAPI/JSON Schema contracts with parity tests; add AsyncAPI when broker consumers are introduced.
- Agno version/model/provider decision record and `AgentOrchestrator` port.
- OTel instrumentation boundary and a deployment-owned collector/dashboard contract.
- Synthetic test district, synthetic contact hierarchy, and fake providers.

Exit criteria:

- A complaint can be created, read, and transitioned through a deterministic test workflow after process restarts.
- A workflow can wait for routing activation, department response, and citizen confirmation without storing those waits in process memory.
- A committed `complaint.received` event can be delivered at least once and start one idempotent Temporal workflow without a request-process side effect.
- Every transition has an event, actor, policy version, and correlation ID.
- A read model can be deleted and rebuilt from events.

Phase 2 — Verified complaint vertical slice

Objective: deliver the core experience against a controlled synthetic execution zone; real MP activation follows the operations-data gate.

Deliverables:

- Native camera-only capture through the greenfield Expo app in `apps/mobile` (no gallery path for verified evidence).
- GPS/accuracy capture and jurisdiction resolution.
- Mandatory voice note for configured categories.
- Voice read-back and tap/icon confirmation.
- Media verification signals and human review queue.
- Deterministic duplicate-candidate and same-area supporter-count boundary; preserve each complaint as an independent record, create only a reviewable cluster link, and use a versioned policy for location-cell/category/time matching. No automatic merge, rejection, priority change, or escalation. This boundary is implemented locally; production exposure remains subject to the product/privacy policy gate.
- Deterministic Tone Governor at the casual-conversation boundary, with neutral safe refusals for political, medical, legal, financial-advice, and explicit threatening-language requests. It must not rewrite complaint facts or bypass the grounded scheme and deterministic filing handlers.
- Agno complaint extraction with Pydantic structured output only.
- Verified routing, synthetic contact hierarchy, and “Mapping in progress” branch.
- Receipt token, redacted public tracking API, and public-safe tracking page.

The public web tracking page, bounded operator control tower, and review-gated citizen browser filing flow are implemented in `apps/web`. Browser evidence has its own signed capture-session and identity policy, is never represented as native attestation, and remains review-required by default. The operator route fails closed without OIDC configuration and never substitutes a shared browser secret for server-side capability checks.

The first mobile implementation deliberately stops at provider boundaries: the capture-attestation adapter rejects production submission until the approved Android/iOS attestation mechanism is installed, and interrupted captures are durably recorded locally with best-effort OS background retry. Multipart retry is implemented at the API and client contract level, and the backend now has a database-claimed cleanup worker for abandoned sessions; signed-device execution, storage CORS, and end-to-end cleanup still require staging validation.

The mobile identity flow uses a generic OIDC account provider for the signed-in session and a server-owned DigiLocker Requester handoff for verification. The app opens the server-generated DigiLocker URL and refreshes status after the browser returns; automatic deep-link completion should be added only after the approved Requester callback behavior is confirmed.

Duplicate handling is deliberately conservative. The first implementation creates an internal candidate cluster from server-owned, policy-approved signals and increments an aggregate supporter count. It never merges complaints, exposes exact locations, or lets an AI handler decide that two reports are the same. Human review, witness confirmation, abuse controls, cluster-level routing, and public collective views are separate follow-on capabilities.

Exit criteria:

- A real complaint can be filed in under two minutes in moderated usability tests.
- The system survives duplicate submissions and retry storms without duplicate complaints.
- A model cannot directly change status, route, priority, or escalation.

Phase 3 — Follow-through automation

Objective: turn a filed complaint into a durable accountability workflow.

Deliverables:

- L1–L4 configuration and contact snapshots.
- Notification adapters and delivery receipts.
- SLA timers, reminders, silence events, weak-reply review, and escalation.
- Department reply ingestion and response classification.
- Private append-only reply records with idempotency and deterministic weak / duplicate signals; reply text does not enter Temporal history or public projections, and a weak signal never resets the SLA by itself.
- Admin control tower and contact-mapping queue.
- MP active-zone runbook and incident procedures.

Exit criteria:

- A simulated no-response complaint escalates through L1–L4 without manual nudging.
- A weak reply is recorded without resetting the underlying SLA.
- Provider outage causes retry/backoff and visible operational alerting.

The local workflow slice now persists the deadline-breach fact before escalation. Live SLA calendars, jurisdiction-owned targets, operational contacts, and public delay projections remain gated by the deferred operations-data decision and the policy actions below.

Phase 4 — Reach and low-literacy expansion

Objective: make the experience usable beyond smartphone text users.

Deliverables:

- WhatsApp adapter with voice/photo flow and provider fake/sandbox contract.
- IVR callback and status flow behind a channel adapter interface.
- SMS/USSD or feature-phone fallback behind an adapter; live telecom activation remains deferred.
- Offline mobile queue and resumable upload.
- Dialect evaluation set and speech fallback.
- Printed/partner receipt option.

Exit criteria:

- Core filing and tracking are completable with voice and tap alone.
- Channel retries and webhook duplicates are idempotent.
- Low-bandwidth usability and failure recovery are tested on representative devices/networks.
- No live telecom channel is enabled until provider, consent, template, and webhook requirements are approved.

Phase 5 — Trust, closure, and transparency

Objective: make public accountability credible rather than merely visible.

Deliverables:

- Proof-based closure and citizen outcomes.
- Witness confirmation and policy-backed collective complaint clusters after the non-destructive candidate boundary has been validated.
- Area memory and chronic-issue views.
- Privacy-safe public dashboard and case pages.
- Case-bundle export worker.
- Weekly/monthly reports.
- Scheme knowledge base and citation-first scheme handler.

Exit criteria:

- Department self-report cannot close a case.
- A citizen can reopen a wrongly closed complaint.
- Public projections contain no unconsented precise location or identity data.
- Scheme answers are traceable to reviewed sources.

Phase 6 — National readiness and scale hardening

Objective: prepare for broad public use and government-backed reliability expectations.

Deliverables:

- Multi-AZ and multi-region recovery design.
- Load, soak, chaos, and disaster-recovery tests.
- Capacity planning for monsoon/crisis spikes.
- Read replicas and partition maintenance.
- GPU/model-cost plan and self-hosted inference pilots.
- CERT-In/security audit readiness and incident drills.
- Expansion playbook for new states/districts.

Exit criteria:

- RTO/RPO and SLOs are tested, not only documented.
 - Expansion is configuration/data work rather than a code fork.
 - Every critical provider has a tested fallback or an explicit operational failure mode.
-

16. Reliability and production requirements

Initial SLOs should be validated through load testing and adjusted with real traffic. They are not marketing promises until measured.

Capability	Initial engineering target
Public web and status reads	99.9% monthly availability at launch; scale toward 99.95%
Complaint command acceptance	p95 under 500 ms excluding media upload and external provider work
Receipt generation	Durable receipt after one successful transactional commit
Event processing	At-least-once delivery with idempotent consumers; no silent loss
Workflow timers	Deadline execution monitored and replay-tested
Media upload	Resumable/chunked where device/network supports it; never proxy large files through API
Database recovery	Multi-AZ backups, point-in-time restore, and tested restore procedure
Initial RPO/RTO	Proposed RPO $\leq$ 5 minutes and RTO $\leq$ 60 minutes; confirm with budget and authority
Privacy	No raw PII in application logs, analytics events, traces, or public projections
Accessibility	Voice/tap completion tested with first-time and low-literacy users, not only automated checks
Cost	AI/media/notification cost recorded per complaint and per channel

Failure behavior

- If the LLM is unavailable, the complaint flow continues with deterministic prompts and a retryable extraction task where possible.
 - If geocoding is unavailable, retain the GPS evidence and enter a review/mapping state; do not invent a jurisdiction.
 - If provider notification fails, record the failure and retry according to policy; do not mark a complaint sent without provider evidence.
 - If a workflow worker is down, Temporal resumes it; the API remains able to show current state.
 - If public projections lag, show a known “data updated at” timestamp rather than presenting stale data as current.
-

17. Security, privacy, and compliance workstream

Security is part of the product design, not a final audit step.

- DPDP-aligned purpose limitation, consent, retention, deletion, access, and grievance processes.
- India-resident storage and processing for citizen data unless an approved exception exists.
- Separate public, citizen, operator, department, moderator, and platform-admin data views.
- Envelope encryption with KMS; secrets only through a managed secret store.
- Private object buckets; short-lived signed access; malware scanning; no public media URLs.
- WAF, API rate limits, device/channel abuse controls, and webhook signature validation.
- Service-to-service identity and least-privilege IAM.
- Dependency pinning, SBOMs, signed images, vulnerability scanning, and protected deployment branches.
- Redacted structured logs with correlation IDs.
- Audit log retention and export integrity verification.
- Incident response plan covering data breach, notification failure, false escalation, and public misinformation.
- Independent security testing before active-zone activation.

CERT-In directions, DPDP obligations, UIDAI rules, telecommunications requirements, WhatsApp terms, and any state-government data-sharing requirements must be confirmed by legal/security owners before production activation. The plan does not treat “government-backed” as permission to bypass provider or statutory controls.

18. Testing and evaluation strategy

Product and domain tests

- State-machine transition tests for every valid and invalid transition.
- Property-based tests for idempotency and duplicate delivery.
- SLA simulation across calendars, time zones, holidays, pauses, replies, and reopenings.
- Routing tests with verified, missing, revoked, and conflicting contacts.
- Privacy projection tests asserting that public APIs cannot return restricted fields.
- Audit reconstruction tests from event history.

Channel tests

- Browser and mobile end-to-end tests with real API contracts, not only mocked responses.
- WhatsApp/IVR webhook replay tests.
- Offline queue and resumable upload tests.
- Low-bandwidth and device capability matrix.

- Accessibility tests using screen readers, voice prompts, keyboard/tap-only interaction, color contrast, and moderated sessions with target users.

AI evaluation

- Intent router confusion matrix.
- Structured extraction accuracy by language, dialect, category, and noisy audio.
- Hallucination tests for scheme answers and officer contacts.
- Political/legal/medical refusal tests.
- Tone neutrality and factual-preservation tests.
- Weak-reply classifier precision/recall with human-labeled cases.
- Retrieval citation correctness and stale-source detection.
- Model drift checks after provider/model changes.

Operational tests

- k6 load tests for intake, tracking, and admin reads.
 - Soak tests for event consumers and workflows.
 - Provider outage, duplicate webhook, delayed delivery, and database failover drills.
 - Disaster-recovery restore and read-model rebuild exercises.
 - Security tests for IDOR, privilege escalation, signed URL misuse, prompt injection, SSRF, malicious media, and webhook forgery.
-

19. First 30 implementation actions

1. Approve the product/legal decisions listed in Section 2.
2. Create the greenfield polyglot monorepo and protect the main branch.
3. Write ADRs for identity, public disclosure, execution zones, SLA ownership, and cloud residency.
4. Define the initial issue taxonomy and MP category SLA table. **Implemented locally:** the versioned launch taxonomy is served by `GET /api/v1/complaints/categories` and consumed by the mobile selector; approved jurisdiction-specific SLA policy remains a production gate.
5. Define the public-safe complaint projection and privacy test fixtures.
6. Define OpenAPI commands/queries and JSON Schema domain events; defer AsyncAPI until a broker consumer exists. **Implemented locally:** `docs/contracts/` is generated by `scripts/export_contracts.py`, parity-tested, and the lifecycle queue validates its versioned payload.
7. Create PostgreSQL/PostGIS migrations and the first aggregate/event schema.
8. Create the OIDC roles/capabilities matrix. **Implemented locally:** see [docs/security/CAPABILITY_MATRIX.md](#); provider registration, assignment review, and staging authorization tests remain activation gates.
9. Create the object-storage bucket policy and upload-token contract.
10. Deploy the Temporal worker and SQS event/outbox workers as separate bounded processes.
11. Add the transactional outbox, idempotency, and optimistic-concurrency middleware.
12. Add OTel tracing and correlation IDs before feature work. **Implemented locally:** the API emits privacy-safe spans/metrics through an OTLP/HTTP boundary; collector deployment and dashboards remain external actions.
13. Build provider fakes for speech, storage, messaging, geocoding, and model calls.
14. Implement the deterministic complaint state machine.
15. Implement the `AgentOrchestrator` port and Agno router/handler spike.
16. Implement the native camera/GPS/voice capture slice.
17. Implement server-side evidence metadata and verification signals.
18. Implement the read-back and consent experience.

19. Implement routing with the verified-contact and mapping-in-progress branches.
 20. Implement receipt generation and public tracking.
 21. Implement L1 workflow timer and a simulated no-response test.
 22. Implement notification provider interfaces and delivery receipts.
 23. Build the admin contact-verification queue. **Implemented locally:** the bounded mapping review queue and capability boundary are present; live operations hierarchy, contacts, calendars, and SLA data are not part of the current implementation. Retain synthetic routing fixtures until activation data is approved.
 24. Add scheme data ingestion and source-review workflow before enabling scheme answers.
 25. Add low-literacy user research with prototypes before polishing visual design.
 26. Add real-provider sandbox contracts only after fake-provider tests pass.
 27. Run a threat model and privacy review on the complete vertical slice. **Implemented locally:** see [docs/security/THREAT_MODEL.md](#).
 28. Run load tests against the real API and database shape; this requires the deployed staging environment and is not replaced by local unit tests.
 29. Run a restore/rebuild drill from backups and events. **Implemented locally:** see [docs/operations/RECOVERY_DRILL.md](#).
 30. Produce an active-zone readiness checklist for Madhya Pradesh and activate automation only after product, security, and operations sign-off. **Future activation prerequisite:** this depends on the deferred live operations data and is not current sprint work.
-

20. Decision register

Decision	Recommendation	Owner / gate
Primary application language	Python for FastAPI, domain, workers, and Agno; TypeScript for web/mobile clients	Engineering
Citizen client	Native Expo app plus web/PWA fallback	Product + mobile engineering
API style	REST/OpenAPI; events for async behavior	Architecture
AI orchestration	Agno behind an AgentOrchestrator port; no second agent framework	Engineering + AI/platform
Workflow engine	Temporal Python SDK for durable complaint/SLA workflows	Architecture + operations
Event delivery	PostgreSQL outbox plus SQS first; Kafka-compatible broker when measured scale requires it	Platform engineering
System of record	PostgreSQL/PostGIS	Data architecture
Media storage	S3-compatible object storage	Security + platform
Identity	OIDC account layer, phone OTP, and DigiLocker Requester verification	Legal + security
Speech/language	Bhashini/provider adapters plus self-hosting evaluation	AI/platform
Cloud reference	AWS India regions with ECS/Fargate launch profile	Finance + security + operations
Initial execution zone	Madhya Pradesh only	Operations/product
Public location precision	Aggregated by policy, never raw by default	Privacy owner
Closure	Proof plus explicit citizen outcome	Product/policy
AI authority	No direct writes to domain state	Architecture/security

21. Action required before live activation

The complete launch-control checklist is maintained separately in [docs/ACTION_REQUIRED.md](#). It contains the external product, privacy, identity, platform, finance, AI, mobile, web, and deployment gates required before live activation, plus the operations-data and telecom items intentionally deferred by the current scope decision.

Engineering can continue locally against the existing ports, migrations, synthetic fixtures, and deterministic fakes while those gates are completed.

22. Definition of done for the redesign baseline

The redesign baseline is complete when:

- the system can accept a verified complaint with explicit evidence and consent;
- state survives restarts and can be reconstructed from events;
- a verified active-zone complaint routes and escalates autonomously through L1–L4;
- an unmapped complaint remains in Mapping in progress without blind dispatch;
- silence and delay are queryable, auditable, and privacy-safe;
- closure requires proof and citizen confirmation;
- all AI outputs are constrained, schema-validated, grounded where required, and unable to mutate critical state directly;
- voice/tap interaction works for non-reading users;
- public transparency is aggregated, neutral, consent-aware, and exportable;
- deployment, recovery, observability, security, and provider-failure behavior have been tested in production-like infrastructure.

This is the point at which the project has a credible production foundation. A working local demo alone is not sufficient evidence of completion.